Object-Oriented Programming

Lecturer:
Teaching Assistant:

Lab 02: Problem Modeling and Encapsulation

In this lab, you will practice with:

- Installing a design tool for UML diagrams: Astah
- Problem Modeling with Use-case diagram
- Encapsulation and different techniques for encapsulation
- Class design for use cases related to cart management
- Java Implementation: Creating classes in Eclipse, constructors, getters and setters, creating instances of classes

0. Assignment Submission

For this lab class, you will have to turn in your work twice, specifically:

- **Right after the class**: for this deadline, you should include any work you have done within the lab class.
- **10PM two days after the class**: for this deadline, you should include the final use case diagram, the final class diagram, the **source code** of all sections of this lab, and the reading assignment in a directory namely "**Lab02**". Note that for the use case diagram, submit both source file (.astah) and its exported image (.png) in the folder namely "**Requirement**"; for the class diagram, also submit both source file (.astah) and its exported image file (.png) in the folder namely "**Design**"; for the reading assignment, submit an image file of the reading assignment in the folder namely "**ReadingAssignment**", put all into a directory namely "**Lab02**" and push it to your **master** branch of the valid repository.

Note that all sample codes or diagrams in the lab are only the examples/suggestions. You may need to change them to satisfy the requirement. Each student is expected to turn in his or her own work and not give or receive unpermitted aid. Otherwise, we would apply extreme methods for measurement to prevent cheating. Please write down answers for all questions into a text file named "**answers.txt"** and submit it within your repository.

1. UML & Astah

The Unified Modeling Language (UML) is a family of graphical notations, backed by single metamodel, that help in describing and designing software systems, particularly software systems built using the object-oriented style (Fowler, 2003).

Astah is a design tool which supports UML. To get Astah UML, go to http://astah.net/student-license-request, fill the form, and send the request. Then follow the 3 steps in the redirected page http://astah.net/student/thank-you. See use case diagram with Astah at http://astah.net/manual/422-usecase-diagram.

Try to draw some several UML diagrams in the Astah UML, such as class diagram, activity diagram, sequence diagram.

2. Problem Statement of AIMS Project

Given the below problem statement of AIMS Project. Please read carefully the statement to ensure that you understand clearly the software that you will design and implement from this lab. If you have any problems, please do not hesitate to ask the professor or TA.

There might be a future that Tiki and Sendo be in talks over a potential merger to contend other e-commerce platforms and especially those who have foreign backers. The merger of these two firms would create a Ti-do company, where "Ti" is from Tiki, and "do" is from Sendo, which means a billion-dollar company in Vietnamese. That firm, Ti-do company, would like you to help them create a brand-new system for AIMS project (AIMS stands for An Internet Media Store). Currently, there is only one type of media: Digital Video Disc (DVD).

Customers can browse the list of DVDs available in the store, the display order is based on their added date, from latest to oldest. When a customer wants to search for DVDs to add to cart, he or she can choose one of three searching options. The software will display a list of all matches (latest DVDs first) with all their information. He or she can also choose to play a specific DVD. The software will play a DVD (a demo part). If a DVD has the length 0 or less, the system must notify the customer that the DVD cannot be played.

- When a customer searches for DVDs by title, he or she provides a string of keywords. If any DVD has the title containing any word in the string of keywords, it is counted as a match. Note that the comparison of words here is case-insensitive.
- When a customer searches for DVDs by category, he or she provides the category name. If any DVD has the matching category (case-insensitive), it is counted as a match.
- When a customer searches for DVDs by price, he or she provides either the minimum and maximum cost, or just the maximum cost.

Customers can view the detail information of a DVD from the list of DVDs. He/she can add a DVD to a cart from a list of DVDs or the detail screen.

When a customer wants to see the current cart, the system displays all the information of the DVDs, along with the total cost. Customers may listen to a DVD (a demo part) in the cart before confirming to place an order. Customers can sort all DVDs in the cart by title or by cost:

- Sort by title: the system displays all the DVDs in the alphabet sequence by title. In case they have the same title, the DVDs having the higher cost will be displayed first.
- Sort by cost: the system the system displays all the DVDs in decreasing cost order. In case they have the same cost, the DVDs will be ordered by increasing title.

Customers can update the quantity of a DVD in a cart or remove a DVD from a cart. To increase consumer demand for the product and grow sales, customers are allowed to have an item for free which is randomly picked out in the cart by the system. Customers can filter DVDs in the cart by providing either its ID or title. If the item is found, display information of the found item in the cart. Or else, notify the customer the item is not found in the current cart.

The customer can request to place order when they are seeing the current cart. For simplification, he/she does not need to log in to place an order. The application will prompt the customer to enter the delivery information and delivery instructions. The software will then calculate the delivery fee based on the total mass of the order & the delivery location. Then, it will display to the customer the invoice including the DVD list, total cost before VAT, total cost after VAT, and the delivery fee. The customer can then proceed to pay for the order. Currently, only one payment method – i.e. credit card – is allowed by connecting to a card association system for checking the validation of the card or performing the pay transaction. After the transaction, the AIMS software will display all the detailed information such as transaction ID, card owner, transaction amount, transaction message, balance, transaction date to the customer. The order will be in pending state and the information of the order & the transaction will be sent to the customer's email.

The store manager needs to log in to the system to navigate to the management mode. He/she can see the list of pending orders, then can pick any order to see its detail to approve or reject the order. The store manager can add new DVDs to the store. He or she must provide all information of the new DVD, including its ID, title, category, director, length, and the cost. Additionally, the manager can also remove DVDs from the store.

3. Use case diagram

Based on the problem statement in the section 2, please draw the use case diagram using Astah UML for the AIMS project.

A use case diagram is one of the UML diagrams to captures dynamic behaviors, i.e., the *pattern of state change over time* (https://www.cs.uct.ac.za/mit_notes/software/htmls/ch05s08.html). The use case diagram illustrates the relations among use cases. A **use-case model** describes a software's functional requirements in terms of use cases. The use-case diagram is a model of the software's intended functions and its environment and serves as a contract between the customer and the developers. Because it is a very powerful planning instrument, the use-case diagram is generally used in all phases of the development cycle. For better understanding, see https://www.uml-diagrams.org/use-case-diagrams.html.

To draw the use case diagram, you must identify the actors and use cases. A sample use case diagram is illustrated in Figure 1 with one use case: place an order. *You have to identify more use cases to update the use case diagram for the final submission.*

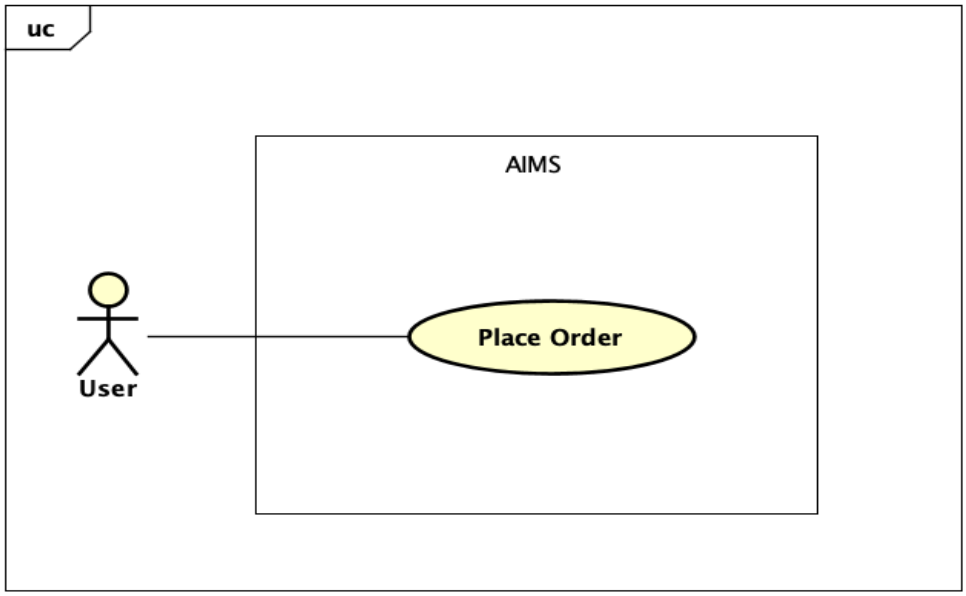

Figure 1. Sample of use case diagram for AIMS Project.

4. UML Class Diagram for use cases related to cart management

The system needs to create a new cart for the user, where it will keep information on the DVDs that the user wants to buy. The user can add, remove DVDs from the cart as well as calculate the cost. The user can add maximum of 20 DVDs into one cart. The cart with its information and behaviors is modeled with the **`Cart`** class. When the user adds a DVD to the cart, the system must also create a new DVD

based on the information that the user provides. This information can be displayed whenever the user decides to see it. The DVD with its information and functions is modeled with the DVD class. Finally, the application needs an entry point for displaying to and taking input from the user (via a command-line interface), which will be the Aims class. A sample class diagram is illustrated in Figure 2, which includes 3 classes:

- The **`Aims`** class which provides a `main()` method which interacts with the rest of the system
- The **`DigitalVideoDisc`** class which stores the title, category, cost, director and length
- The **`Cart`** class to maintain an array of these `DigitalVideoDisc` objects

You have to update this class diagram following the below exercises for the final submission.

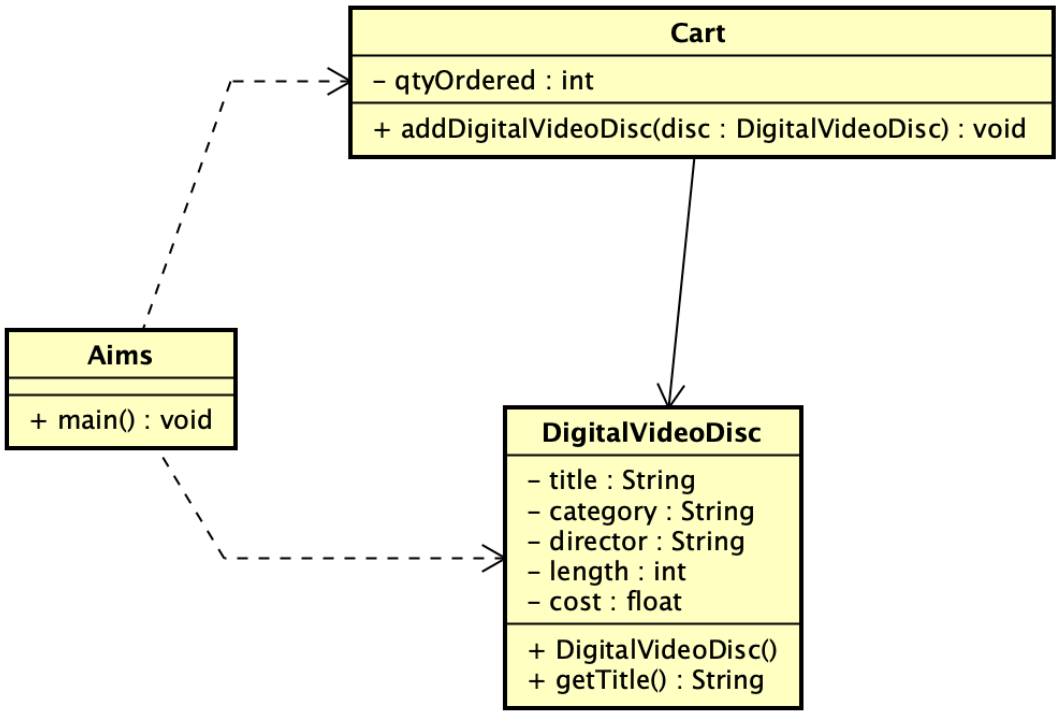

Figure 2. Sample class diagram for use cases related to cart management.

5. Create Aims class

- Open Eclipse

- Create a new JavaProject named "AimsProject"

- Create Aims class: In the src folder, create a new class named Aims:

+ Right click on the folder and choose New -> Class:

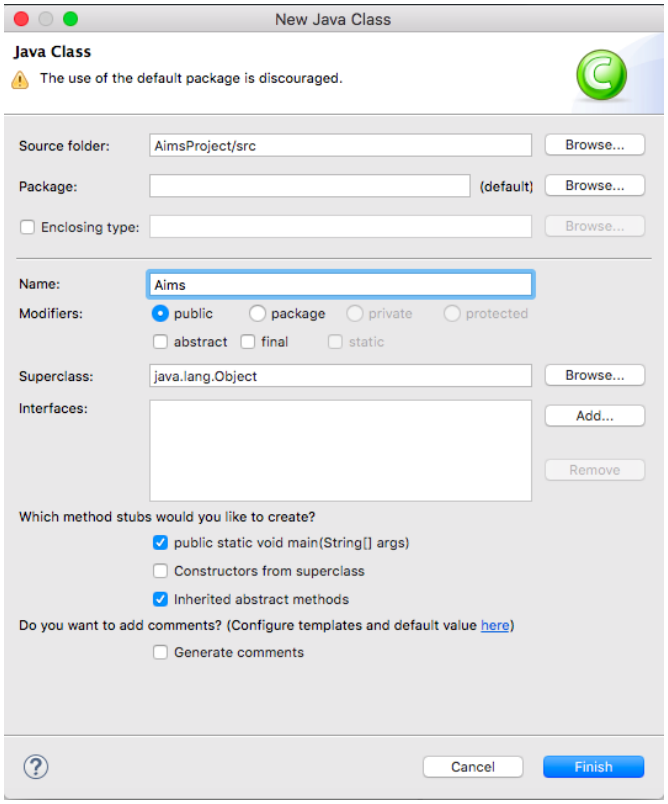

Figure 3. Create Aims class by Eclipse

+ You may need to check the option "`public static void main(String[] args)`" This will automatically generate the main function in the class Aims.java as the following result.

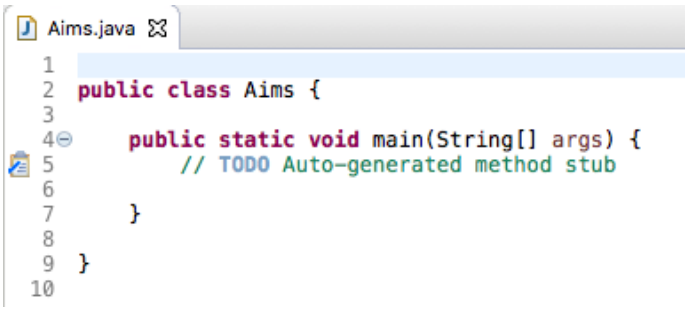

Figure 4. Generated code for Aims class

+ Because you did not choose any package for the Aims class, Eclipse then displays the icon package and mentions (default package) for your class.

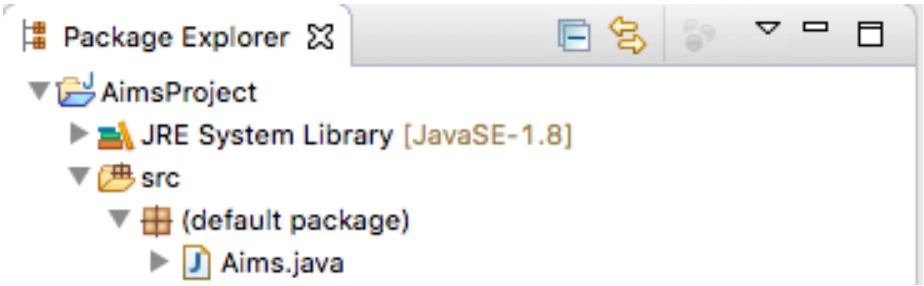

Figure 5. Aims in default package

+ You can create a package and move the class to this package if you want. In the folder scr, a sub-folder will be created (with the name of the package) to store the class. Do it yourself and open the src folder to see the result.

6. Create the DigitalVideoDisc class and its attributes

Make sure that the option for the main method is not checked.
Open the source code of the DigitalVideoDisc class and add some attributes below:

```
Aims.java    DigitalVideoDisc.java

1
2  public class DigitalVideoDisc {
3      private String title;
4      private String category;
5      private String director;
6      private int length;
7      private float cost;
8
9      |
10 }
11
```

Figure 6. DigitalVideoDisc class

7. Create accessors and mutators for the class DigitalVideoDisc

To create setters and getters for private attributes, you can create methods to allow controlled public access to each of these private variables. Eclipse allows you to do this automatically. However, in many cases, you do not allow to create accessors and mutators for all attributes, but depending on the business. E.g. in a bank account, the balance cannot be modified directly through a mutator, but should be increased or decreased through credit or debit use cases.

- **Right click anywhere in the source file of DigitalVideoDisc.**
- **Choose Source, then choose Generate Getters and Setters** (Figure 7)
- **Choose the attributes that needs getters/setters**
    - **For each of them, choose the dropdown arrow next to the tick box and choose to generate only setter, getter, or both.**
      <u>Suggestion</u>: To choose the appropriate getters/setters, one should examine carefully the requirements of the system. In the case of Aims, based on the description of the system, we can decide the appropriate accessor methods for each attribute of the DVD class as follows: Firstly, there is no use case that requires the change of the attributes of a DVD after it is added, so we eliminate all the **setters**. Secondly, since the system needs to display all information of the DVDs when the user sees the current cart, all the **getters** are chosen. (Figure 8)
- **Choose the option "public" in the Access modifier**
- **Click Generate**

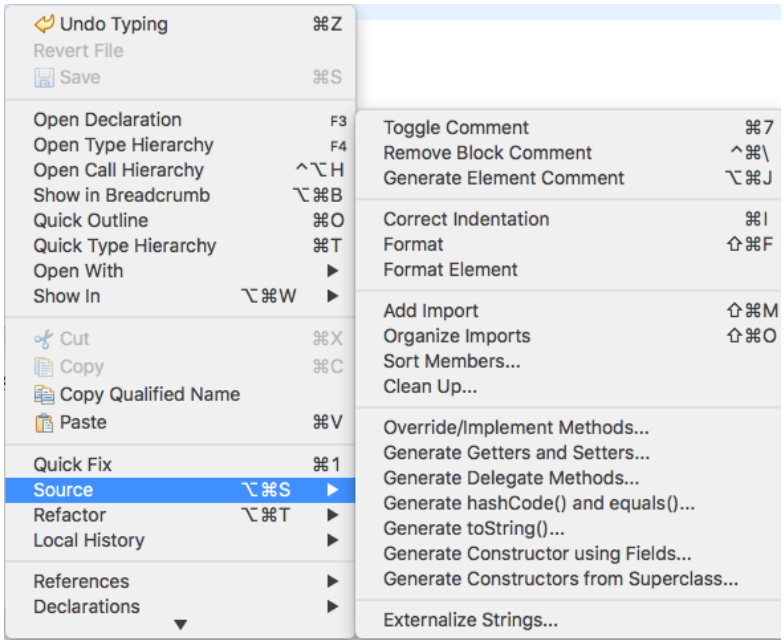

Figure 7. Generate getters & setters by Eclipse

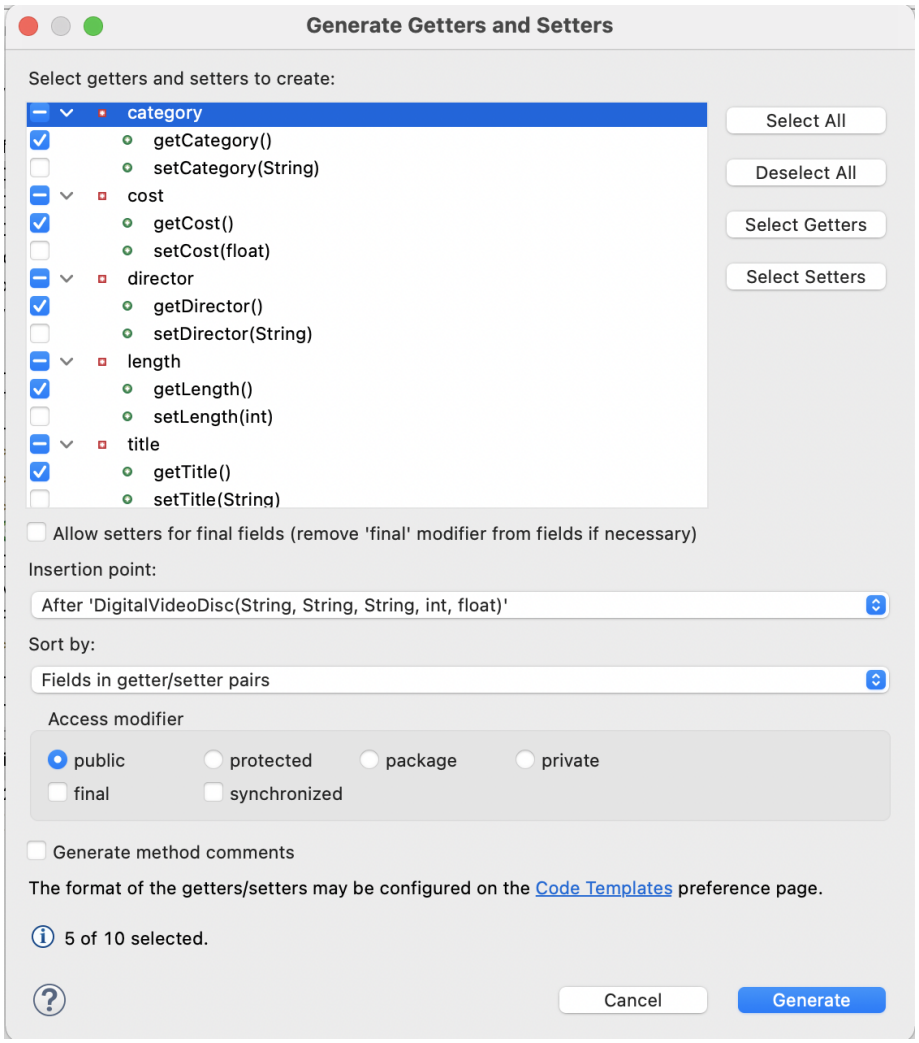

Figure 8. Choose appropiate accessors

```java
public String getTitle() {
    return title;
}
public String getCategory() {
    return category;
}
public String getDirector() {
    return director;
}
public int getLength() {
    return length;
}
public float getCost() {
    return cost;
}
```

Figure 9. Generated accessors

Reading Assignment: When should accessor methods be used?
Read the following article and find the best possible answer to the above question: Holub, Allen. "Why getter and setter methods are evil" *JavaWorld*, 5 Sep. 2003, https://www.infoworld.com/article/2073723/why-getter-and-setter-methods-are-evil.html
You should expand your research to other sources as well. For the response, give a summary of your findings in the form of a mindmap. You can draw this mind map by hand and take a picture of your work or use any online tools. In both cases, the accepted format for the image file is one of the following: .png, .jpg, .jpeg and .pdf.

8. Create Constructor method

By default, all classes of Java will herit from the **`java.lang.Object`**. If a class has no constructor method, this class in fact uses the constructor method of **`java.lang.Object`**. Therefore, you can always create an instance of class by a no-arguments constructor method. For example:

```java
DigitalVideoDisc dvd1 = new DigitalVideoDisc();
```

In this part, you will create yourself constructor method for DigitalVideoDisc for different purposes:
- Create a DVD object by title
- Create a DVD object by category, title and cost
- Create a DVD object by director, category, title and cost
- Create a DVD object by all attributes: title, category, director, length and cost

Each purpose will be corresponding to a constructor method. By doing that, you have practiced with overloading method.

Question:

- If you create a constructor method to build a **DVD** by title then create a constructor method to build a **DVD** by category. Does JAVA allow you to do this?

Eclipse also allows you to automatically generate constructor methods by field. Just do the same as generating getters and setters. Right click anywhere in the source file, Choose Source, Choose Generate constructors by fields (Figure 10) then select the fields (Figure 11) to generate constructor methods.

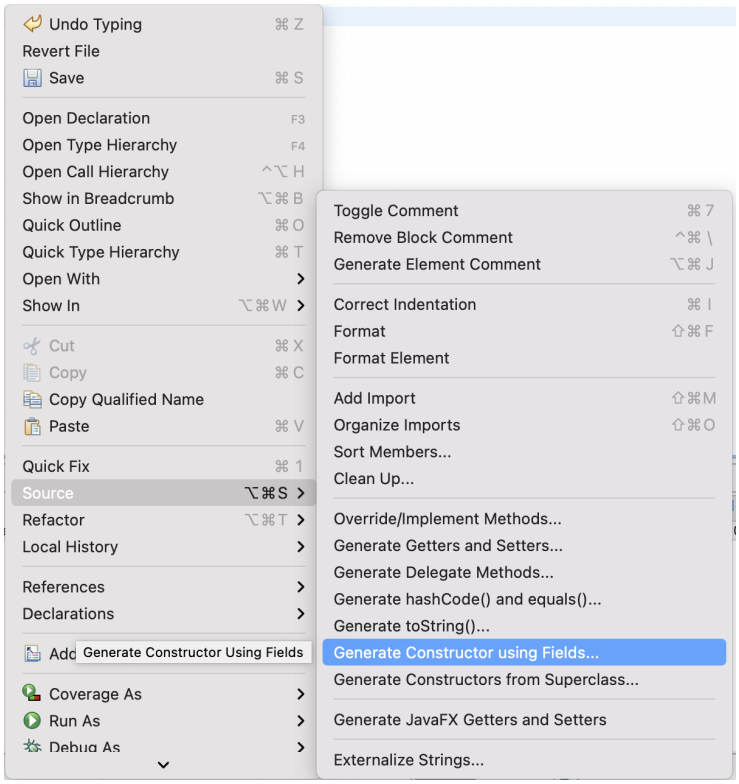

Figure 10. Generating constructor using fields

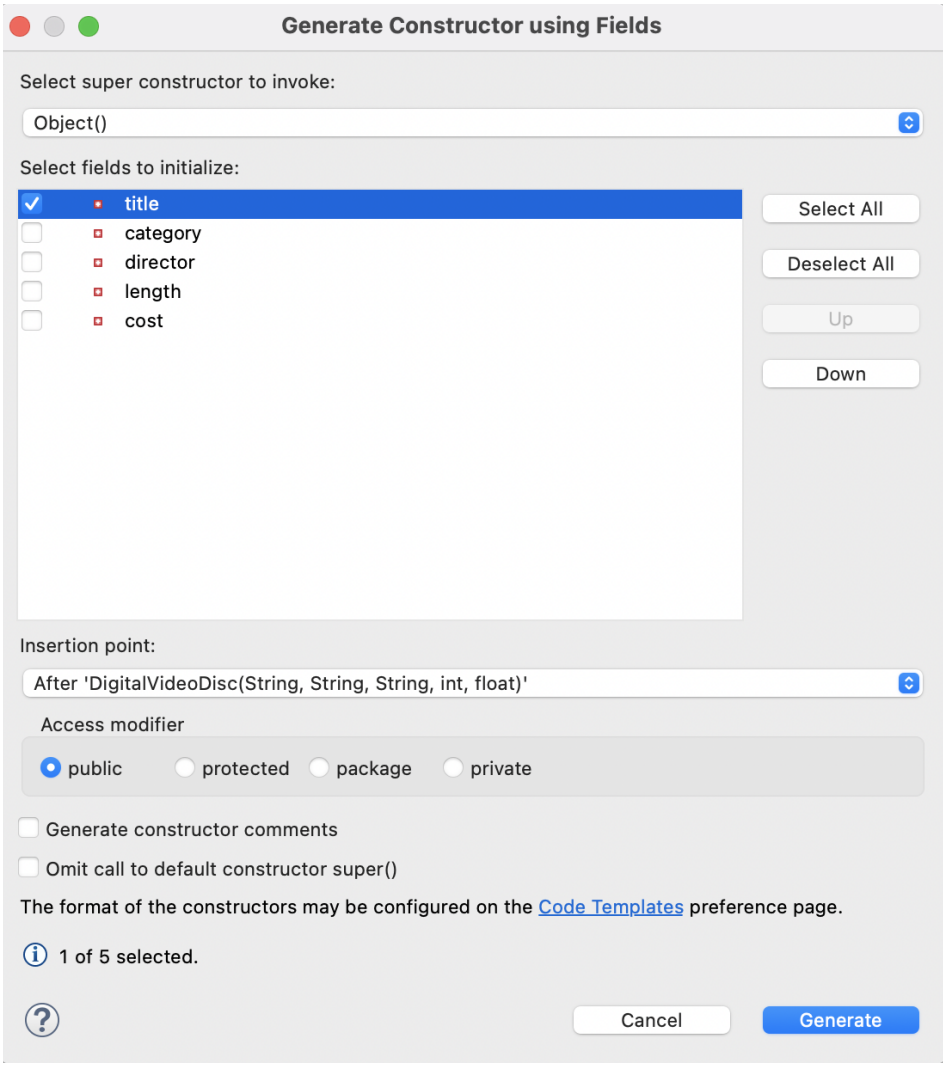

Figure 11. Setup for generating constructor using fields

The result is:

```
public DigitalVideoDisc(String title) {
    super();
    this.title = title;
}
```

Figure 12. A sample code for a generated constructor

This is how you create the first constructor method. Similarly, you will create by yourself the others.

9. Create the Cart class to work with DigitalVideoDisc

The **`Cart`** class will contain a list of **`DigitalVideoDisc`** objects and have methods capable of modifying the list.

```
public class Cart {

    public static final int MAX_NUMBERS_ORDERED = 20;
    private DigitalVideoDisc itemsOrdered[] =
            new DigitalVideoDisc[MAX_NUMBERS_ORDERED];

}
```

Figure 13. Sample code of the Cart class

Add a field as an array to store a list of **`DigitalVideoDisc`**.
To keep track of how many DigitalVideoDiscs are in the cart, you must create a field named **`qtyOrdered`** in the Cart class which stores this information.
Create the method **`addDigitalVideoDisc(DigitalVideoDisc disc)`** to add an item to the list. You should check the current quantity to assure that the cart is not already full

- Create the method **`removeDigitalVideoDisc(DigitalVideoDisc disc)`** to remove the item passed by argument from the list.

Create the **`totalCost()`** method which loops through the values of the array and sums the costs of the individual **`DigitalVideoDiscs`**. This method returns the total cost of the current cart.
Note that your methods should interact with users. For example: after adding it should inform the user: "**`The disc has been added`**" or "**`The cart is almost full`**" if the cart is full.
Now you have all the classes for the application. Just practice with them in the next section.

10. Create Carts of DigitalVideoDiscs

The **`Aims`** class should create a new Cart, and then create new DVDs and populate the cart with those DVDs. This will be done in the **`main()`** method of the Aims class.
Do the following code in your main method and run the program to test.

```java
public static void main(String[] args) {

    //Create a new cart
    Cart anOrder = new Cart();

    //Create new dvd objects and add them to the cart
    DigitalVideoDisc dvd1 = new DigitalVideoDisc("The Lion King",
            "Animation", "Roger Allers", 87, 19.95f);
    anOrder.addDigitalVideoDisc(dvd1);

    DigitalVideoDisc dvd2 = new DigitalVideoDisc("Star Wars",
            "Science Fiction", "George Lucas", 87, 24.95f);
    anOrder.addDigitalVideoDisc(dvd2);

    DigitalVideoDisc dvd3 = new DigitalVideoDisc("Aladin",
            "Animation", 18.99f);
    anOrder.addDigitalVideoDisc(dvd3);

    //print total cost of the items in the cart
    System.out.println("Total Cost is: ");
    System.out.println(anOrder.totalCost());

}
```

Figure 14. Sample code of the Aims class

The result should be:

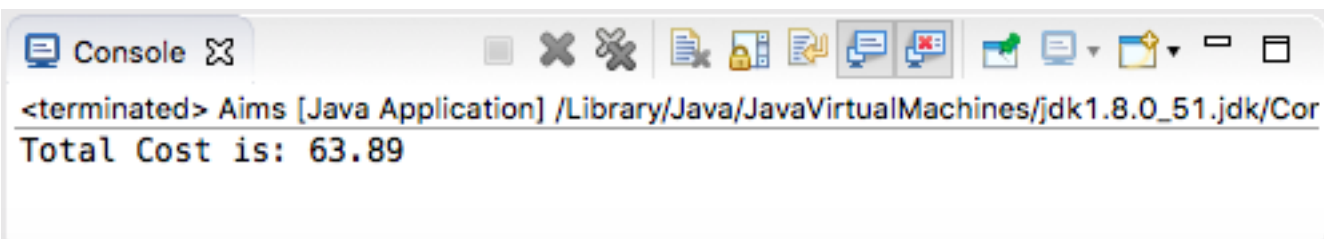

Figure 15. Results for creating a cart of digital video discs.

11.Removing items from the cart

You have to write code in your main method to test the **`removeDigitalVideoDisc(DigitalVideoDisc disc)`** method of the Cart class and check if the code is successfully run.

12. References

James Rumbaugh, Ivar Jacobson, and Grady Booch (2004). *Unified Modeling Language Reference Manual, The (2nd Edition)*. Pearson Higher Education